ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 25

Controladores, vistas y modelos

- La configuración en un .env
- El modelo base: un mini ORM
- Repository o Active Record
- El motor de vistas y las vistas parciales
- El controlador base
- Excepciones HTTP: 403, 404 y 405
- Las siete rutas de un recurso
- El controlador de recurso



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Nuestro MVC ya sabe recibir peticiones, encontrar la ruta y devolver una respuesta. Pero solo sabe **leer**. Hoy le damos el CRUD completo, y por el camino convertimos cada una de las tres letras de MVC en una pieza reutilizable del *framework*:

- un **controlador base** del que heredan todos los controladores;
- un **motor de vistas** con vistas parciales;
- un **modelo base**, un pequeño ORM al estilo de Eloquent, el de Laravel;
- y la configuración sensible, fuera del código, en un archivo `.env`.

Cuando terminéis, el `CursosControlador` tendrá las siete acciones típicas de un recurso, igual que un controlador de Laravel.

•
👌 Cómo vamos a trabajar



1
⚠️ La configuración en un .env



2
📘 El modelo base: un mini ORM



3
🤔 Repository o Active Record



4
🔍 El motor de vistas y las vistas parciales



5
🚀 El controlador base



6
🐞 Excepciones HTTP: 403, 404 y 405



7
👌 Las siete rutas de un recurso



8
😮 El controlador de recurso



9
🚀 Ejercicios



10
👋 Lo que tenéis que llevaros de hoy





CLASE 25

👌 Cómo vamos a trabajar

Sustituid el contenido de `academia-mvc` por el de `proyecto/`, copiad `.env.ejemplo` **como** `.env` y ejecutad `composer dump-autoload`.

Esquema

```
.env.ejemplo                  ← NUEVO: plantilla de configuración (se sube a Git)
.env                          ← NUEVO: vuestra configuración (NO se sube a Git)
src/Nucleo/
├── Entorno.php               ← NUEVO: lee el .env
├── BaseDatos.php             ← NUEVO: sustituye a Conexion
├── Modelo.php                ← NUEVO: el modelo base (mini ORM)
├── Vista.php                 ← NUEVO: el motor de vistas
├── Controlador.php           ← NUEVO: el controlador base
├── Sesion.php                ← la de la clase 21, dentro del núcleo
├── HttpException.php         ← NUEVO
└── Peticion, Respuesta, Router y ManejadorErrores
src/Modelos/Curso.php         ← ahora hereda de Modelo
vistas/cursos/                ← index, ver, crear, editar y _formulario
```

Desaparecen `src/BaseDatos/`, `src/Repositorios/` y `src/Excepciones/`: sus funciones las asumen las piezas nuevas.



CLASE 25 · PUNTO 1

⚠️ La configuración en un .env

Hasta ahora, el usuario y la contraseña de la base de datos estaban **escritos en el código**, en `config/config.php`. Eso trae dos problemas: si subís el proyecto a GitHub, subís las contraseñas; y en el servidor son distintas que en vuestro ordenador, así que tendríais que cambiar el código al publicar.

La solución que usa todo el mundo, Laravel incluido, es un archivo `.env` con las **variables de entorno**: lo que cambia según el ordenador [1].

Esquema

```
APP_NOMBRE="Academia Alpe"
APP_DEPURACION=true

DB_HOST=localhost
DB_NOMBRE=academia_crud
DB_USUARIO=root
DB_CLAVE=
```

Archivo	Qué es	¿Se sube a Git?
`.env`	Vuestra configuración real, con contraseñas	❌ **Nunca** (está en `.gitignore`)
`.env.ejemplo`	Una plantilla con los nombres de las variables, sin secretos	✅ Sí, para que cada uno sepa qué tiene que rellenar

Para leerlo, una clase sencilla que recorre el archivo línea a línea:

Entorno.php — php-paso-a-paso/clase25 — Visual Studio Code

Entorno.php

```php
<?php
class Entorno
{
    private static array $variables = [];

    public static function cargar(string $archivo): void
    {
        foreach (file($archivo, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES) as $linea) {
            $linea = trim($linea);
            if ($linea === "" || str_starts_with($linea, "#")) {
                continue;
            }
            [$nombre, $valor] = array_map("trim", explode("=", $linea, 2) + [1 => ""]);
            self::$variables[$nombre] = trim($valor, "\"'");
        }
    }

    public static function obtener(string $nombre, string $porDefecto = ""): string
    {
        return self::$variables[$nombre] ?? $porDefecto;
    }
}
```

Y `BaseDatos::conexion()` toma de ahí sus datos: `Entorno::obtener("DB_USUARIO")`.

📄 Archivos: `src/Nucleo/Entorno.php`, `src/Nucleo/BaseDatos.php`, `.env.ejemplo` y `config/config.php`



💡 Laravel usa una librería muy conocida para esto, `vlucas/phpdotenv`, que hace lo mismo con más opciones. Nosotros escribimos la nuestra para ver que no hay ningún misterio.



CLASE 25 · PUNTO 2

📘 El modelo base: un mini ORM

En la clase 21, cada tabla necesitaba su repositorio con sus consultas. Con tres tablas, tres repositorios casi iguales: `buscar`, `guardar` y `borrar` con distinto nombre de tabla. ¿Os suena? Falta una clase padre.

El **modelo base** escribe el SQL **una sola vez**, para cualquier tabla. Cada modelo hijo solo dice cómo se llama su tabla y qué columnas tiene [2]:

Modelo.php — php-paso-a-paso/clase25 — Visual Studio Code

Modelo.php

```php
<?php
class Curso extends Modelo
{
    protected static string $tabla = "cursos";
    protected static array $columnas = [
        "titulo" => "string",
        "horas" => "int",
        "precio" => "float",
        "plazas" => "int",
    ];

    public string $titulo = "";
    public int $horas = 20;
    public float $precio = 0;
    public int $plazas = 20;

    // ...y SU lógica: errores(), precioConIva(), estaCompleto()
}
```

Y con eso, el curso ya sabe hacer todo esto:

Código	Qué hace	SQL que genera el modelo base
`Curso::todos()`	Todos los cursos	`SELECT * FROM cursos ORDER BY id`
`Curso::buscar(5)`	El curso 5, o `null`	`SELECT * FROM cursos WHERE id = :id`
`Curso::paginaDonde("titulo", "php", 1, 5)`	Una página de resultados	`... WHERE titulo LIKE :texto LIMIT … OFFSET …`
`(new Curso())->rellenar($_POST)`	Rellena las propiedades, convirtiendo los tipos	—
`$curso->guardar()`	Sin id: `INSERT`; con id: `UPDATE`	`INSERT INTO cursos (titulo, horas…) VALUES (:titulo, :horas…)`
`$curso->borrar()`	Borra la fila	`DELETE FROM cursos WHERE id = :id`

¿Cómo sabe el modelo base en qué tabla trabajar? Con `static::` : dentro de `Modelo`, `static::$tabla` es la tabla **de la clase hija** desde la que se llama. Es el *enlace estático tardío* que apareció en el temario como «para saber más»: `self::` es «esta clase» y `static::` es «la clase con la que me han llamado» [3].

Modelo.php — php-paso-a-paso/clase25 — Visual Studio Code

Modelo.php

```php
<?php
public static function buscar(int $id): ?static
{
    $consulta = static::pdo()->prepare("SELECT * FROM " . static::$tabla . " WHERE id =
:id");
    $consulta->execute(["id" => $id]);
    $fila = $consulta->fetch();
    return $fila === false ? null : static::desdeFila($fila);
}
```

El tipo de retorno `?static` significa «un objeto de la clase hija, o `null`»: `Curso::buscar(5)` devuelve un `Curso`.



🛡️ **Atentos a la seguridad.** Los nombres de tabla y columna **no** pueden ir en un marcador (`:columna`). Por eso el modelo base solo acepta columnas que estén en `$columnas` (`columnaPermitida()`): la lista blanca de la clase 16, ahora dentro del *framework*.

📄 Archivos: `src/Nucleo/Modelo.php` y `src/Modelos/Curso.php`



CLASE 25 · PUNTO 3

🤔 Repository o Active Record

Habéis visto ya las dos formas más habituales de organizar el modelo:

	Repository (clase 21)	Active Record (hoy)
¿Quién sabe SQL?	Una clase aparte: `CursoRepositorio`	El propio objeto, gracias a su clase padre
Buscar	`$repositorio->buscarPorId(5)`	`Curso::buscar(5)`
Guardar	`$repositorio->guardar($curso)`	`$curso->guardar()`
Ventaja	Separación total entre datos y base de datos	Muy cómodo y con mucho menos código
Lo usa...	Symfony (Doctrine) y proyectos grandes	**Laravel (Eloquent)**

Ninguna es «la buena»: son dos estilos. Como el siguiente paso del curso es Laravel, a partir de hoy seguimos con Active Record.



CLASE 25 · PUNTO 4

🔍 El motor de vistas y las vistas parciales

La función `vista()` de la clase 23 se convierte en la clase `Vista`, con dos métodos:

Método	Qué hace
`Vista::mostrar("cursos/index", $datos)`	Genera la vista **dentro del layout**
`Vista::parcial("cursos/_formulario", $datos)`	Genera un **trozo** de vista, sin layout

Las **vistas parciales** son trozos que se repiten en varias vistas. El formulario de un curso, por ejemplo, es el mismo al crear y al editar (os suena de la clase 16). Por convenio, su nombre empieza por guion bajo:

Vista.php — php-paso-a-paso/clase25 — Visual Studio Code

Vista.php

```
1 <h1>➕ Nuevo curso</h1>
2 <?= parcial("cursos/_formulario", ["curso" => $curso, "errores" => $errores, "accion" =>
  "/cursos"]) ?>
```

`parcial()` y `campoCsrf()` son dos funciones de atajo para las vistas, en `src/funciones.php`. Igual que en Laravel, donde escribís `@include('cursos.formulario')` y `@csrf`.

📄 Archivos: `src/Nucleo/Vista.php`, `vistas/cursos/_formulario.php`, `crear.php` y `editar.php`



Todos los controladores repiten lo mismo: devolver una vista, redirigir con un mensaje, responder con un 404… Lo ponemos en un **controlador base** abstracto del que heredan:

Controlador.php — php-paso-a-paso/clase25 — Visual Studio Code

Controlador.php

```php
<?php
abstract class Controlador
{
    protected function vista(string $nombre, array $datos = [], int $codigo = 200):
Respuesta
    {
        return Respuesta::html(Vista::mostrar($nombre, $datos), $codigo);
    }

    protected function redirigir(string $destino, ?string $mensaje = null, string $tipo =
"ok"): Respuesta
    {
        if ($mensaje !== null) {
            Sesion::guardarMensaje($mensaje, $tipo);
        }
        return Respuesta::redirigir($destino);
    }

    protected function noEncontrado(string $mensaje = "No encontramos lo que buscas."):
never
    {
        throw new HttpException(404, $mensaje);
    }

    protected function comprobarCsrf(Peticion $peticion): void { /* lanza HttpException(403)
si no es válido */ }
}
```

Fijaos en `noEncontrado()`, con tipo `never`: no devuelve nada, **lanza** una excepción. Eso permite escribir esta línea tan cómoda:

Controlador.php — php-paso-a-paso/clase25 — Visual Studio Code

Controlador.php

```php
<?php
$curso = Curso::buscar($id) ?? $this->noEncontrado("Ese curso no existe.");
```

«Busca el curso; si es `null`, responde con un 404». Una línea, sin `if`.

📄 Archivo: `src/Nucleo/Controlador.php`



CLASE 25 · PUNTO 6

🐞 Excepciones HTTP: 403, 404 y 405

`HttpException` es una excepción que **ya sabe con qué código responder**:

HttpException.php — php-paso-a-paso/clase25 — Visual Studio Code

HttpException.php

```php
<?php
class HttpException extends RuntimeException
{
    public function __construct(public readonly int $codigo, string $mensaje = "")
    {
        parent::__construct($mensaje);
    }
}
```

La lanzan el controlador base (403 por CSRF, 404 por `noEncontrado()`) y el router (404 y 405). El manejador de errores las reconoce y muestra la vista `errores/{código}` con el mensaje, sin tratarlas como un fallo del servidor:

HttpException.php — php-paso-a-paso/clase25 — Visual Studio Code

HttpException.php

```php
<?php
if ($e instanceof HttpException) {
    $html = Vista::mostrar("errores/{$e->codigo}", ["titulo" => "Error {$e->codigo}",
"mensaje" => $e->getMessage()]);
    Respuesta::html($html, $e->codigo)->enviar();
    return;
}
```

📄 Archivos: `src/Nucleo/HttpException.php` y `src/Nucleo/ManejadorErrores.php`



💡 Laravel tiene la función `abort(404)`, que hace exactamente esto.



CLASE 25 · PUNTO 7

👌 Las siete rutas de un recurso

Un **recurso** es algo que se puede listar, ver, crear, editar y borrar: un curso, un alumno, un producto. Para cada recurso se usan siempre las mismas siete rutas, con los mismos nombres de acción [4]:

Método	Ruta	Acción	Para qué
GET	`/cursos`	`index`	Listado
GET	`/cursos/crear`	`crear`	Formulario vacío
POST	`/cursos`	`guardar`	Guardar el nuevo
GET	`/cursos/{id}`	`ver`	Ficha
GET	`/cursos/{id}/editar`	`editar`	Formulario relleno
POST	`/cursos/{id}`	`actualizar`	Guardar los cambios
POST	`/cursos/{id}/borrar`	`borrar`	Borrar

rutas.php — php-paso-a-paso/clase25 — Visual Studio Code

rutas.php

```php
<?php
$router->get("/cursos", [CursosControlador::class, "index"]);
$router->get("/cursos/crear", [CursosControlador::class, "crear"]);
$router->post("/cursos", [CursosControlador::class, "guardar"]);
$router->get("/cursos/{id:\d+}", [CursosControlador::class, "ver"]);
$router->get("/cursos/{id:\d+}/editar", [CursosControlador::class, "editar"]);
$router->post("/cursos/{id:\d+}", [CursosControlador::class, "actualizar"]);
$router->post("/cursos/{id:\d+}/borrar", [CursosControlador::class, "borrar"]);
```

Fijaos en `{id:\d+}`: la restricción del ejercicio 02 de la clase 24, ya incorporada al router. Gracias a ella, `/cursos/crear` no se confunde con `/cursos/{id}`.

📄 Archivo: `config/rutas.php`



💡 En Laravel, estas siete rutas se crean con una sola línea: `Route::resource('cursos', CursoController::class)`. Las acciones se llaman `index`, `create`, `store`, `show`, `edit`, `update` y `destroy`, y en lugar de POST usan PUT y DELETE para actualizar y borrar. Lo veremos en la API REST.



CLASE 25 · PUNTO 8

😮 El controlador de recurso

Así queda la acción `guardar()`. Comparadla con el `crear.php` de la clase 21:

CursosControlador.php — php-paso-a-paso/clase25 — Visual Studio Code

CursosControlador.php

```php
<?php
// POST /cursos
public function guardar(Peticion $peticion): Respuesta
{
    $this->comprobarCsrf($peticion);
    $curso = (new Curso())->rellenar($peticion->todoElFormulario());
    $errores = $curso->errores();

    if (count($errores) > 0) {
        return $this->vista("cursos/crear", ["titulo" => "Nuevo curso", "curso" => $curso,
"errores" => $errores], 422);
    }

    $curso->guardar();
    return $this->redirigir("/cursos/{$curso->id}", "Curso «{$curso->titulo}» creado.");
}
```

Dos detalles:

- Si hay errores, la respuesta lleva el código **422** (*Unprocessable Content*): «he entendido lo que me mandas, pero los datos no son válidos». Es el que usa Laravel en estos casos.
- Tras guardar, redirigimos a la **ficha** del curso nuevo, que ya tiene id gracias a `guardar()`.

📄 Archivo: `src/Controladores/CursosControlador.php`

✅ Pruebas para comprobarlo

Haced esto	Debéis obtener
`/`	Redirige a `/cursos`
`/cursos?q=php` y `/cursos?pagina=3`	Búsqueda y paginación
Crear un curso con título «AB» y 0 horas	El formulario con los errores (código 422)
Crear un curso correcto	Su ficha, con el mensaje verde
Editar un curso	El formulario relleno; al guardar, la ficha actualizada
Borrar «PHP 8 desde cero»	«No se puede borrar: el curso tiene alumnos»
Borrar el curso que habéis creado	Vuelta al listado con el mensaje
`/cursos/99` y `/cursos/abc`	404
Abrir `/cursos/2/borrar` en la barra de direcciones	405



⚠️ Ahora mismo **cualquiera** puede crear, editar y borrar cursos: no hay login. Lo arreglamos en la clase 26 con un *middleware* de autenticación.



CLASE 25 · PUNTO 9

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Modelo Alumno.** Cread el modelo `Alumno` (tabla `alumnos`: nombre, email y `curso_id`), un `AlumnosControlador` con la acción `index` y la ruta `GET /alumnos`. ¿Cuántas líneas de SQL habéis escrito? → `Alumno.php`, `AlumnosControlador.php` y `vistas/alumnos/index.php`
- ☐ **02 · Relaciones.** Añadid al modelo base un método `donde($columna, $valor)` y usadlo para dos relaciones: `$alumno->curso()` (un alumno pertenece a un curso) y `$curso->alumnos()` (un curso tiene muchos alumnos). Mostrad los alumnos en la ficha del curso. → `Modelo.php`, `Curso.php` y `vistas/cursos/ver.php`
- ☐ **03 · Una variable más.** Añadid `APP_CORREO` al `.env` (y al `.env.ejemplo`) y mostradlo en el pie de todas las páginas. → `config/config.php` y `vistas/layout.php`



💡 En el modelo `Alumno`, la propiedad se llama `curso_id`, igual que la columna, en lugar de `cursoId`. Es la excepción a nuestra norma de *camelCase*: así `rellenar()` la encuentra sin traducir nombres. Laravel hace lo mismo con sus modelos.



🏆 **Reto:** cread el CRUD completo de alumnos con sus siete rutas. En el formulario, un `<select>` con los cursos (`Curso::todos("titulo")`). Comprobad lo poco que tenéis que escribir ahora.



CLASE 25 · PUNTO 10

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`.env`	La configuración que cambia según el ordenador, fuera del código y fuera de Git
Modelo base	El SQL de `buscar` , `guardar` y `borrar` escrito una vez para todas las tablas
`static::`	«La clase con la que me han llamado»: así el modelo base sabe la tabla
Active Record	`Curso::buscar(5)` y `$curso→guardar()` : el estilo de Eloquent
Vistas parciales	Trozos de vista reutilizables: `parcial("cursos/_formulario")`
Controlador base	`vista()` , `redirigir()` , `noEncontrado()` y `comprobarCsrf()` para todos
`HttpException`	Una excepción que sabe su código: 403, 404, 405
Recurso	Siete rutas y siete acciones: index, crear, guardar, ver, editar, actualizar y borrar

📚 Fuentes y documentación oficial

1. The Twelve-Factor App: III. Configuración
2. Martin Fowler: Active Record
3. Manual de PHP: enlaces estáticos en tiempo de ejecución (static::)
4. Laravel: Resource Controllers